

Face Effects

1. Experiment Objective

Use the dlib 68-point facial landmark detection model to add AR effects such as eye filling, lip beautification and eyebrow highlighting to the face.

2. Experiment Principle

1. Key Terms

Keyword	Description
dlib 68 points	The classic 68 facial landmark model, covering eyebrows (10 points), eyes (12 points), nose (9 points), lips (20 points) and jawline (17 points)
Eye filling	Extracts an ROI from the eye contour landmarks (36-47) and fills the pupil region with a semi-transparent color
Lip beautification	Extracts an ROI from the lip contour landmarks (48-67) and applies color overlay and blur for beautification

2. Technical Architecture

```
Terminal 1: start agent + camera + joint models
Terminal 2: start face effects
```

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☒ Supported
No-PTZ version	☒ Supported
Required peripherals	ESP32 camera

2. Start the agent + camera + joint models (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```

yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-10-47-55-372666-yahboom-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[micro_ros_agent-1] [1785206877.173080] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046238399] [esp32_ip_camera_publisher]: IP camera stream opened successfully

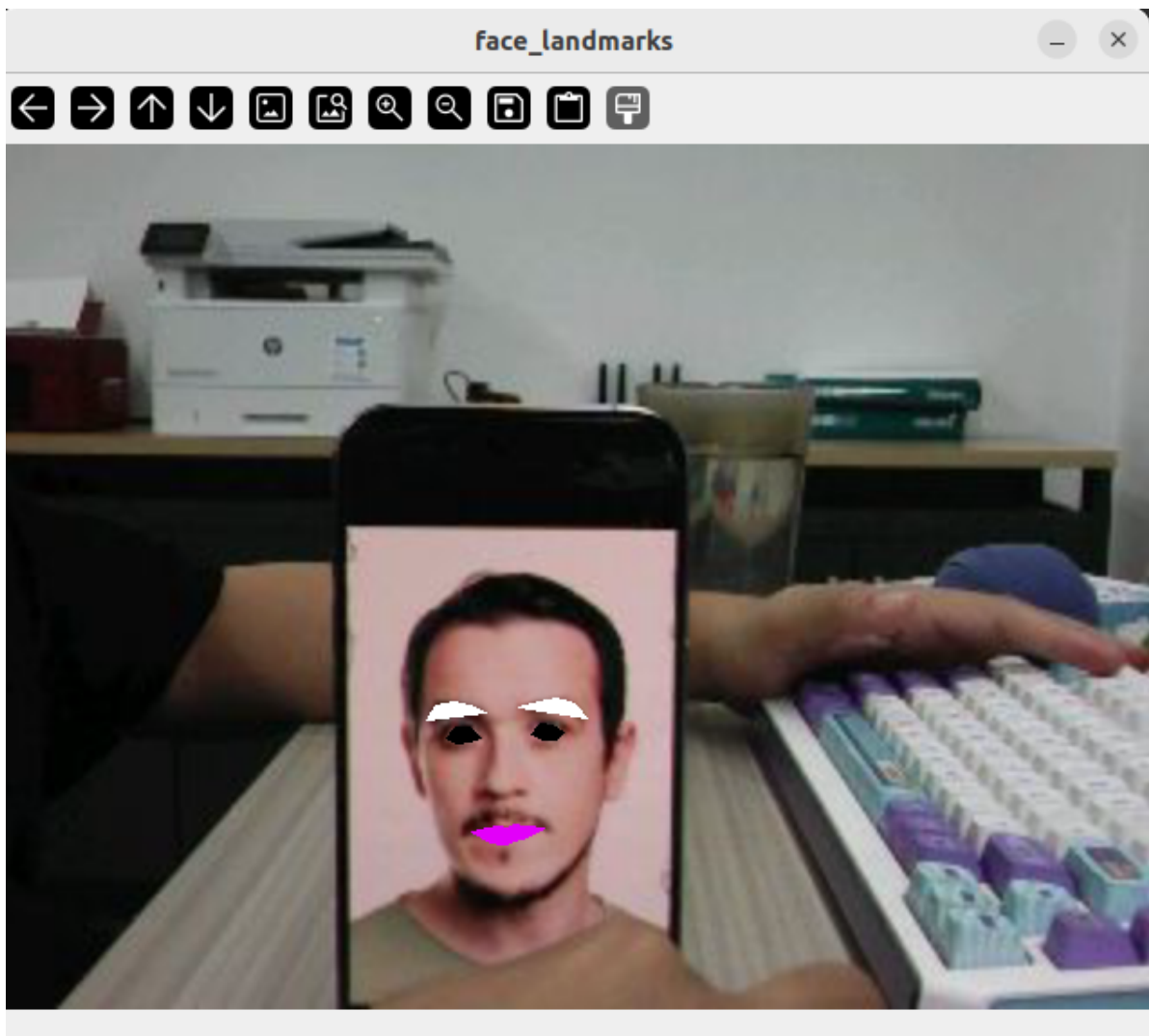
```

3. Start face effects (Terminal 2)

```
ros2 launch microros_v2_mediapipe_pkg face_landmarks.launch.py
```

4. Observed Results

- The face is annotated with 68 cyan landmarks
- The eye region is filled with semi-transparent blue, and the lip region is beautified with a pink overlay
- The eyebrow region is highlighted



5. Code Walkthrough

Source code paths:

- Launch:
`~/microros_ws/src/microros_v2_mediapipe_pkg/launch/face_landmarks.launch.py`
- Node:
`~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/FaceLandmarks.py`

1. Core Node Analysis

```
# Source file:
~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/FaceLandmarks.py

class FaceLandmarks(Node):
    def __init__(self):
        super().__init__("face_landmarks_node")
        self.detector = dlib.get_frontal_face_detector()
        self.predictor =
dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")

    def image_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
        faces = self.detector(gray)
        for face in faces:
            landmarks = self.predictor(gray, face)
            # Eye ROI (landmarks 36-47)
            eye_pts = np.array([[landmarks.part(n).x, landmarks.part(n).y]
                                for n in range(36, 48)], np.int32)
            cv.fillPoly(overlay, [eye_pts], (255, 0, 0))
            # Lip ROI (landmarks 48-67)
            lip_pts = np.array([[landmarks.part(n).x, landmarks.part(n).y]
                                for n in range(48, 68)], np.int32)
            cv.fillPoly(overlay, [lip_pts], (255, 0, 255))
            frame = cv.addWeighted(frame, 0.7, overlay, 0.3, 0)
            cv.imshow(self.window_name, frame)
```

Key logic:

- `dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")`: loads the 68-point model file
- Eye landmark indices 36-47 (12 points, 6 per eye); lip indices 48-67 (20 points)
- `cv.fillPoly` draws filled polygons on the overlay layer, and `cv.addWeighted` blends them with the original image at 0.3 opacity
- dlib 68 points vs MediaPipe Face Mesh 468 points: dlib is lighter and starts faster; the 468-point mesh is finer but more computationally expensive

2. Key Parameters

Parameter definition files:

- Launch:
`~/microros_ws/src/microros_v2_mediapipe_pkg/launch/face_landmarks.launch.py`
- Node:
`~/microros_ws/src/microros_v2_mediapipe_pkg/microros_v2_mediapipe_pkg/FaceLandmarks.py`

Parameter	Type	Default	Description
<code>predictor_path</code>	string	<code>shape_predictor_68_face_landmarks.dat</code>	Path to the 68-point model file

3. Node Description

Node	Function
<code>face_landmarks_node</code>	dlib 68-point detection + eye/lip/eyebrow effect overlay

6. Frequently Asked Questions

Issue	Cause	Solution
Model fails to load	The 68-point .dat file is missing	Verify that <code>shape_predictor_68_face_landmarks.dat</code> exists